Контест 4 · Базы данных и кэш

Неделя 8 · темы недель 7–8: SQL и связи таблиц, рейтинг Эло, миграции Alembic, типы данных Redis, TTL и очереди. Сдача устная у экрана: показываете код, запускаете на примерах, объясняете идею.

> Монеты: обычная задача — **2**, со звёздочкой (★, например B4*) — **4**, каждая часть босса — **6**. В этом контесте: 15 обычных, 3 со звёздочкой, 3 частей босса — всего **60** монет. Задача засчитывается, только если вы можете объяснить свой код.

A. SQL

Во всех задачах A — база SQLite (`import sqlite3`) со схемой ниже. Ответ получайте **одним SQL-запросом** (для A4* — запрос плюс короткий цикл в Python). Параметры подставляйте через `?`, а не f-строкой.

```
CREATE TABLE players (id INTEGER PRIMARY KEY, name TEXT UNIQUE NOT NULL, rating INTEGER NOT
NULL DEFAULT 1000);
CREATE TABLE matches (id INTEGER PRIMARY KEY, p1 INTEGER NOT NULL REFERENCES players(id),
                      p2 INTEGER NOT NULL REFERENCES players(id),
                      result TEXT NOT NULL CHECK (result IN ('win1', 'win2', 'draw')));
```

Пример данных для всех задач A:

```
players: 1 Аня 1100 · 2 Боря 1050 · 3 Вика 1100 · 4 Гоша 1000
matches (p1, p2, result): (1, 2, win1) (2, 3, draw) (3, 1, win2) (1, 2, win2) (1, 3, win1)
(1, 2, win1)
```

A1. `top_players(conn, n)` — первые `n` игроков: рейтинг по убыванию, при равенстве — по имени.

```
Ввод:  top_players(conn, 3)
Вывод: [("Аня", 1100), ("Вика", 1100), ("Боря", 1050)]
```

A2. `games_count(conn)` — сколько партий сыграл каждый игрок (за любую сторону). Игроки без партий тоже нужны.

```
Ввод:  games_count(conn)
Вывод: {"Аня": 5, "Боря": 4, "Вика": 3, "Гоша": 0}
```

A3. `win_rate(conn, name)` — доля побед игрока, округлённая до двух знаков; нет партий или нет игрока — `None`.

```
Ввод:  win_rate(conn, "Аня"), win_rate(conn, "Боря"), win_rate(conn, "Гоша")
Вывод: 0.8, 0.25, None
```

A4*. `best_streak(conn, name)` — самая длинная серия побед подряд (партии по порядку `id`; ничья серию прерывает).

```
Ввод:  best_streak(conn, "Аня")
Вывод: 2
```

B. Рейтинг

Ожидаемый счёт `E = 1 / (1 + 10 ** ((rb − ra) / 400))`, изменение `Δ = round(k · (score − E))`, `k = 32`.

B1. `elo_delta(ra, rb, score, k=32)` — изменение рейтинга игрока A (у B — ровно `−Δ`).

```
Ввод:  elo_delta(1000, 1000, 1), elo_delta(1200, 1000, 1), elo_delta(1000, 1200, 1),
elo_delta(1200, 1000, 0.5)
Вывод: 16, 8, 24, -8
```

B2. `replay(matches, start=1000)` — проиграйте партии `(a, b, result)` по порядку, начиная со `start` для каждого нового игрока. Верните итоговые рейтинги, отсортированные по имени.

```
Ввод:  replay([("Аня", "Боря", "win1"), ("Боря", "Вика", "draw"), ("Аня", "Вика", "win2")])
Вывод: {"Аня": 999, "Боря": 985, "Вика": 1016}
```

B3. `rank_with_ties(ratings)` — места с учётом равенства «как на олимпиаде»: 1, 1, 3, 4. Внутри места — по имени.

```
Ввод:  rank_with_ties({"Аня": 1100, "Боря": 1050, "Вика": 1100, "Гоша": 1000})
Вывод: [(1, "Аня"), (1, "Вика"), (3, "Боря"), (4, "Гоша")]
```

C. Redis

Функции асинхронные и получают клиент `r` (`redis.asyncio.Redis` с `decode_responses=True`; в тестах — `fakeredis`). Подберите подходящий тип данных — это часть задачи.

C1. `add_win(r, name)` добавляет игроку победу и возвращает их число; `top_wins(r, n)` — первые `n` по победам, при равенстве по имени. Ключ — `wins`.

```
Ввод:  победы: Аня, Боря, Аня, Вика, Боря, Аня;  top_wins(r, 3)
Вывод: [("Аня", 3), ("Боря", 2), ("Вика", 1)]
```

C2. `remember_recent(r, user, item, limit=5)` — «недавние игры»: `item` встаёт первым, повторов нет, хранится не больше `limit`. Возвращает список после добавления. Ключ — `recent:<user>`.

```
Ввод:  по очереди c4, snake, c4, chess, go, ttt, mines
Вывод: ["mines", "ttt", "go", "chess", "c4"]
```

C3. `once(r, key, ttl)` — защита от двойной записи результата партии: `True` только при первом вызове с этим ключом в течение `ttl` секунд. Ключ — `once:<key>`. Одна команда Redis.

```
Ввод:  once(r, "match:42", 60), once(r, "match:42", 60), once(r, "match:43", 60)
Вывод: True, False, True
```

C4*. `sliding_allow(r, key, limit, window, now)` — скользящее окно: разрешить, если за последние `window` секунд (интервал `(now − window, now]`) было меньше `limit` **разрешённых** запросов. `now` передаётся явно.

```
Ввод:  limit=3, window=10, now = 0, 1, 2, 3, 9.5, 10.5, 11, 12, 13
Вывод: True, True, True, False, False, True, True, True, False
```

D. Схемы и миграции

D1. `parse_ddl(sql)` — по тексту с командами `CREATE TABLE` (регистр любой, бывает `IF NOT EXISTS`) верните словарь «таблица → список колонок». Строки-ограничения (`PRIMARY KEY (...)`, `UNIQUE`, `FOREIGN KEY`, `CHECK`, `CONSTRAINT`) — не колонки. Осторожно: запятые бывают внутри скобок, `NUMERIC(6, 1)`.

```
Ввод:  CREATE TABLE players (id INTEGER PRIMARY KEY, tg_id BIGINT NOT NULL,
           rating NUMERIC(6, 1) DEFAULT 1000, UNIQUE (tg_id));
Вывод: {"players": ["id", "tg_id", "rating"]}
```

D2. `migration_order(revisions)` — словарь «ревизия → предыдущая» (`None` у первой), как `down_revision` в Alembic. Верните порядок применения. Две ревизии с одной предыдущей (ветвление), ссылка на несуществующую ревизию, цикл или несколько первых — `ValueError`.

```
Ввод:  migration_order({"c3": "b2", "a1": None, "b2": "a1"})
Вывод: ["a1", "b2", "c3"]
```

D3. `review_migration(text)` — автоматическое ревью файла миграции. Верните предупреждения по порядку: `"drop"` — в `upgrade` есть `op.drop_column` или `op.drop_table`; `"not_null_without_default"` — `op.add_column` с `nullable=False` без `server_default` (каждый вызов в одной строке); `"empty_downgrade"` — в `downgrade` нет ни одного `op.`.

```
Ввод:  def upgrade() -> None:
           op.drop_column("players", "rating")
           op.add_column("players", sa.Column("elo", sa.Integer(), nullable=False))

       def downgrade() -> None:
           pass
Вывод: ["drop", "not_null_without_default", "empty_downgrade"]
```

E. Проект и алгоритмы

E1. PostgreSQL в compose вашего проекта: покажите `docker compose ps` (у `db` статус `healthy`), `alembic upgrade head` на чистой базе и `\d matches` в `psql`. Данные переживают `docker compose down` и `up`.

E2. Индекс на деле. Заполните таблицу `matches` 200 000 строк, покажите план и время запроса `... WHERE p1 = ? ORDER BY id DESC LIMIT 10` до и после индекса (`EXPLAIN QUERY PLAN` в SQLite или `EXPLAIN ANALYZE` в PostgreSQL). Объясните, почему индекс на `(p1, id)` лучше, чем на `p1`.

E3. Хранение комнат: попросите агента предложить два варианта (например, hash на комнату или JSON-строка; комнаты в Redis или в PostgreSQL) и составьте с командой таблицу «плюсы / минусы» по 4+ критериям. Защитите выбранный вариант.

E4 · босс. Турнир по вашей игре. За каждую часть свой плюс.

а) `round_robin(players)` — расписание «каждый с каждым» методом круга: первый игрок стоит на месте, остальные сдвигаются по кругу на одну позицию каждый тур. В туре пары `(players[i], players[n-1-i])`. При нечётном числе добавьте `None` в конец — его соперник отдыхает (пара `(игрок, None)`).

```
Ввод:  round_robin(["A", "B", "C", "D"])
Вывод: [[("A", "D"), ("B", "C")], [("A", "C"), ("D", "B")], [("A", "B"), ("C", "D")]]
```

б) `standings(players, results)` — таблица `(имя, очки, победы, ничьи, поражения)`: победа 3 очка, ничья 1. Порядок: очки, победы — по убыванию, затем имя. Неизвестный игрок или результат — `ValueError`.

```
Ввод:  standings(["Аня", "Боря", "Вика", "Гоша"],
                 [("Аня", "Боря", "win1"), ("Боря", "Вика", "draw"), ("Вика", "Аня",
"win1")])
Вывод: [("Вика", 4, 1, 1, 0), ("Аня", 3, 1, 0, 1), ("Боря", 1, 0, 1, 1), ("Гоша", 0, 0, 0,
0)]
```

в) `swiss_pairs(order, played)` — пары «швейцарского» тура: игроки по месту в таблице; первый свободный играет с ближайшим по таблице, с кем ещё не встречался (`played` — множество `frozenset` пар). Если дальше разбить нельзя — перебор с возвратом. Невозможно или число игроков нечётное — `ValueError`.

```
Ввод:  swiss_pairs(["A", "B", "C", "D"], {frozenset("CD")})
Вывод: [("A", "C"), ("B", "D")]          # A–B, C–D нельзя: C и D уже играли
```

E5*. Класс `LRU(capacity)` — кэш, который при переполнении выбрасывает то, к чему дольше всего не обращались. `get(key, default=None)`, `put(key, value)`, `len()`, `keys()` (от давнего к свежему), счётчики `hits` и `misses`. Все операции — O(1). Подсказка: `collections.OrderedDict.move_to_end`.

```
Ввод:  c = LRU(2); c.put("a", 1); c.put("b", 2); c.get("a"); c.put("c", 3); c.keys()
Вывод: ["a", "c"]
```